



Ministerio del Poder Popular para la Educación

Universidad Nacional Experimental de Guayana

Ingeniería en Informática

Unidad Curricular: Lenguajes y Compiladores

Sección 2

Tema 5: Análisis Sintáctico

Profesor:

Marquez Felix

Estudiantes:

Cortez Samuel V27.158.842

Duarte Alejandro V28.530.657

Moreno Alondra V30.331.870

Perdomo Arnaldo V30.791.551

Ciudad Guayana, Julio 2026

Índice

Introducción.....	4
Teoría de compiladores y lenguajes UNEGSCRIPT.....	5
1 Árbol de Sintaxis Abstracta.....	5
1.1 Definición Formal y Características.....	5
1.2. Dos (2) Ejemplos de Entrada expresadas en AST.....	5
1.2.1 Ejemplo 1: Expresión Aritmética.....	5
1.2.2 Ejemplo 2: Estructura de Control (Condicional).....	7
1.3. Código / Implementación Conceptual del AST (Python).....	8
2. Análisis Sintáctico: Comparación entre Gramáticas y Parsers LL vs LR.....	9
2.1 Cuadro Comparativo.....	9
2.2. Evidencia de Uso Actual en la Industria.....	10
2.3 Dos Ejemplos Prácticos e Implementación (Mismo Lenguaje L).....	10
Ejemplo LL(1): Parser Recursivo Descendente (Python).....	10
Ejemplo LR(1) / Shift-Reduce Simplificado (Python).....	10
3.Resumen de Generadores de Analizadores Sintácticos (Metacompiladores).....	10
3.1. Metacompiladores Principales.....	11
3.2 Cuadro Comparativo entre Metacompiladores Principales.....	12
3.3 Código / Asistente Híbrido (Python).....	13
3.4 Técnicas de Gestión y Recuperación de Errores Sintácticos.....	15
5. Implementación en UnegScript.....	17
5.1 Módulo Lexer Híbrido (Regex + Fallback LLM / Levenshtein).....	17
5.2 Código de Implementación del Lexer Híbrido (Python).....	18

6. Descripción del trabajo realizado.....	20
6.1 Especificación común (tokens y gramática).....	20
6.2 Implementaciones desarrolladas.....	20
6.3 Conjunto de prueba.....	21
6.4 Script de benchmarking.....	21
6.5 Generación de gráficas.....	21
7 Gráficas de tiempo de ejecución.....	22
7.1 Tiempo promedio general por lenguaje/herramienta.....	22
7.2 Tiempo promedio por lenguaje (escala logarítmica).....	22
7.3 Tiempo de parseo en función del tamaño del archivo.....	23
7.4 Variabilidad de las mediciones.....	23
7.5 Resumen numérico.....	24
8. Discusión y análisis de resultados.....	25
8.1 Diferencias generales entre lenguajes.....	25
8.2 El caso de Java: overhead de arranque de la JVM.....	25
8.3 Relación entre tamaño del archivo y tiempo de ejecución.....	26
8.4 Variabilidad de las mediciones.....	26
8.5 Conclusiones del experimento.....	26
Conclusión.....	28
Bibliografía.....	30

Introducción

El desarrollo de software moderno y el diseño de lenguajes de programación requieren una profunda comprensión de cómo se procesa, analiza y ejecuta el código fuente. En el ámbito de la teoría de compiladores, el análisis léxico y sintáctico constituye la columna vertebral para la transformación de texto plano en estructuras ejecutables y semánticamente válidas.. Este informe final consolida el trabajo teórico y práctico realizado en la asignatura Lenguajes y Compiladores de la Universidad Nacional Experimental de Guayana (UNEG), a incluir desde las bases conceptuales del lenguaje de procesamiento hasta la implementación de soluciones avanzadas y la evaluación de rendimiento en entornos reales..

A lo largo del documento se abordan tres ejes fundamentales. En primer lugar, se explora la teoría de compiladores mediante el estudio de los Árboles de Sintaxis Abstracta (AST), la comparación analítica entre métodos de parseo LL y LR, y el funcionamiento de los metacompiladores e hiperherramientas de análisis.. En segundo lugar, se presenta la especificación formal e implementación práctica del lenguaje *UnegScript* , integrando un módulo de análisis léxico y sintáctico híbrido que aprovecha algoritmos de distancia de cadenas y modelos de lenguaje (IA) para la recuperación automática de errores.. Finalmente, se documenta un experimento de *benchmarking* multilenguaje (desarrollado en Python, Java y Rust) enfocado en el análisis de archivos *Dockerfile* , evaluando el impacto de la arquitectura de cada lenguaje y su modelo de ejecución sobre la eficiencia computacional.

Teoría de compiladores y lenguajes UNEGSCRIPT

1 Árbol de Sintaxis Abstracta

1.1 Definición Formal y Características

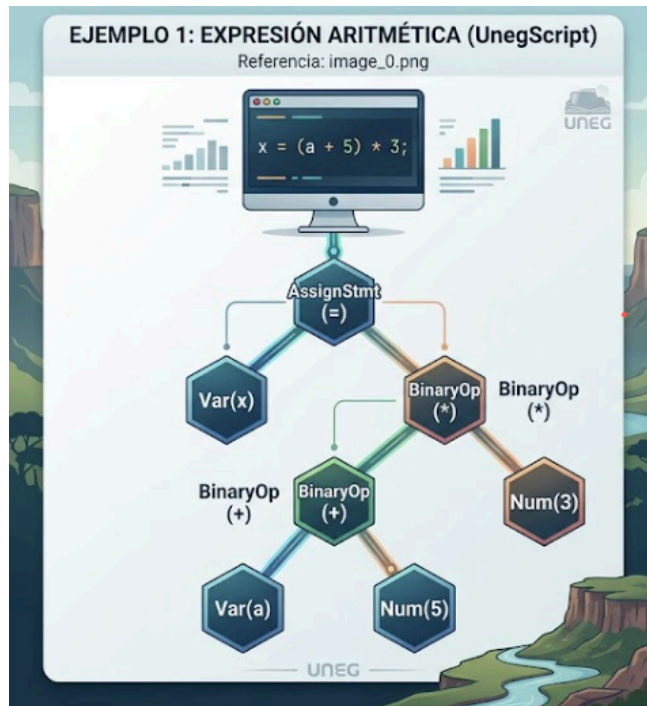
Un **Árbol de Sintaxis Abstracta (AST)** es una representación en forma de árbol n-ario de la estructura sintáctica abstracta del código fuente de un programa.

- **¿Por qué es "Abstracto"?** A diferencia del Árbol de Derivación Sintáctica (o *Parse Tree*), el AST elimina detalles sintácticos irrelevantes de la gramática como delimitadores, paréntesis, puntos y comas o palabras claves repetitivas, conservando únicamente la semántica de la instrucción.
- **Características Principales:**
 - **Nodos Internos:** Representan operadores, instrucciones de control o llamadas a funciones (ej: **+**, **if**, **while**).
 - **Nodos Hoja:** Representan operandos o identificadores (ej: variables, constantes numéricas).
 - **Jerarquía de Operaciones:** Refleja explícitamente la precedencia y asociatividad de los operadores sin necesidad de paréntesis.

1.2. Dos (2) Ejemplos de Entrada expresadas en AST

1.2.1 Ejemplo 1: Expresión Aritmética

- **Código de Entrada:** $x = (a + 5) * 3;$
- **Representación en AST:**



Código de entrada: $x = (a + 5) * 3;$

Explicación teórica y estructural:

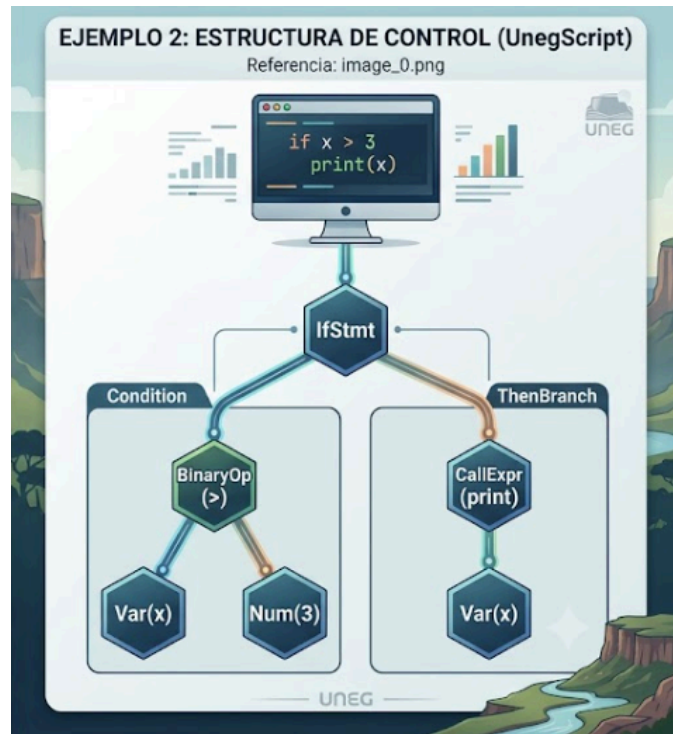
Nodo Raíz (AssignStmt): Representa la operación principal del enunciado, que es una asignación (=). Tiene dos componentes directos: el identificador destino a la izquierda (x) y la expresión evaluada a la derecha.

Simplificación de la paréntesis: En el código fuente, la suma (a + 5) está encerrada entre paréntesis para alterar la precedencia de operadores. En el AST, **los paréntesis desaparecen por completo** . La prioridad de ejecución se representa de forma natural mediante la jerarquía del árbol: la suma se sitúa en un nivel inferior (hijo izquierdo) con respecto al producto . **BinaryOp (+)** **BinaryOp (*)**

Evaluación descendente/ascendente: Para calcular el valor que tomará x, el compilador primero evalúa el subárbol más profundo (a + 5), luego multiplica el resultado por la hoja **Num(3)** y finalmente asigna el valor resultante a la hoja **Var(x)**.

1.2.2 Ejemplo 2: Estructura de Control (Condicional)

- **Código de Entrada:** `if x > 3 print(x)`
- **Representación en AST:**



Código de entrada: `if x > 3 print(x)`

Explicación teórica y estructural:

Nodo Raíz (IfStmt): Modela la sentencia de control condicional. A diferencia de un nodo aritmético básico, un nodo ifactúa como una estructura contenedora que organiza la lógica del flujo del programa en dos o tres ramificaciones principales (Condición, Rama Then, y opcionalmente Rama Else).

Rama Condicional (Condition): Su hijo es un nodo de comparación binaria **BinaryOp (>)** que evalúa si la variable **Var(x)** es mayor que el valor constante **Num(3)**. Esta rama devuelve un valor booleano (True/ False).

Rama de Ejecución (ThenBranch): Contiene la instrucción que se ejecutará si la condición resulta verdadera. En este caso, apunta a un nodo de llamada a función CallExpr (print), el cual tiene como parámetro a la variable Var(x).

Abstracción léxica: Al igual que en el caso anterior, palabras clave como if o paréntesis de la función print() dejan de ser tokens individuales en una lista para transformarse en una representación semántica estructurada donde las dependencias son explícitas.

1.3. Código / Implementación Conceptual del AST (Python)

```
class ASTNode:
    """Clase base para todos los nodos del AST en UnegScript"""
    pass

class NumNode(ASTNode):
    def __init__(self, value):
        self.value = value

    def __repr__(self):
        return f"NumNode({self.value})"

class VarNode(ASTNode):
    def __init__(self, name):
        self.name = name

    def __repr__(self):
        return f"VarNode('{self.name}')"

class BinaryOpNode(ASTNode):
    def __init__(self, left, op, right):
        self.left = left      # ASTNode
        self.op = op          # str ('+', '*', '>', etc.)
        self.right = right    # ASTNode

    def __repr__(self):
        return f"BinaryOpNode({self.left}, '{self.op}', {self.right})"
```



```

class AssignNode(ASTNode):
    def __init__(self, var_name, expr):
        self.var_name = var_name # str
        self.expr = expr # ASTNode

    def __repr__(self):
        return f"AssignNode('{self.var_name}', {self.expr})"

class IfNode(ASTNode):
    def __init__(self, condition, then_branch, else_branch=None):
        self.condition = condition # ASTNode
        self.then_branch = then_branch # ASTNode
        self.else_branch = else_branch # ASTNode

    def __repr__(self):
        return f"IfNode(Cond: {self.condition}, Then: {self.then_branch}, Else: {self.else_branch})"

```

2. Análisis Sintáctico: Comparación entre Gramáticas y Parsers LL vs LR

2.1 Cuadro Comparativo

Criterio	Análisis LL (Left-to-right, Leftmost derivation)	Análisis LR (Left-to-right, Rightmost derivation in reverse)
Complejidad	Baja. Se puede implementar fácilmente mediante analizadores recursivos descendentes manuales.	Alta. Requiere el cálculo de conjuntos FIRST/FOLLOW y autómatas LR(0)/LR(1), generando tablas de gran tamaño.
Flexibilidad	Restictiva. No admite recursión izquierda (directa o indirecta) ni ambigüedades en gramáticas.	General/Alta. Soporta gramáticas más amplias y complejas, incluyendo expresiones anidadas y asociatividad.
Tablas	Pequeñas y simples (filas = no terminales, columnas = terminales).	Grandes estados del autómata LR vs. terminales/no terminales).
Gramáticas	Subconjunto GLC: LL(k), principalmente LL(1).	Subconjunto GLC más amplio: LR(0), SLR(1), LALR(1), LR(1).

2.2. Evidencia de Uso Actual en la Industria

- **Analizadores LL (Descendentes):**

- **GCC y Clang (C/C++):** Utilizan parsers recursivos descendentes escritos a mano en lugar de generadores automáticos, para ofrecer diagnósticos detallados de errores sintácticos y optimizaciones durante el parseo.
- **TypeScript / Rust:** Sus compiladores oficiales emplean analizadores descendentes manuales para controlar el estado del análisis y facilitar el reporte claro de errores al programador.
- **ANTLR:** Framework ampliamente usado en la industria que genera analizadores LL(*).

- **Analizadores LR (Ascendentes):**

- **Yacc / Bison:** Utilizados históricamente y en producción para definir lenguajes mediante sintaxis LALR(1) (por ejemplo, partes del parser de GCC tradicional o procesadores de JSON/SQL).
- **GLR (Generalized LR):** Usado en herramientas de análisis sintáctico multilenguaje para analizar sintaxis complejas y ambiguas como C++ legacy o COBOL.

2.3 Dos Ejemplos Prácticos e Implementación (Mismo Lenguaje L)

Consideremos el lenguaje elemental L de expresiones aritméticas sencillas compuestas por números (num) e incrementos (+).

Ejemplo LL(1): Parser Recursivo Descendente (Python)

Para LL(1), eliminamos la recursión izquierda reescribiendo la gramática:

$$E \rightarrow \text{num } E'$$
$$E' \rightarrow + \text{num } E' \mid \epsilon$$

Ejemplo LR(1) / Shift-Reduce Simplificado (Python)

La gramática LR permite recursión izquierda directa:

$$E \rightarrow E + \text{num} \mid \text{num}$$

3. Resumen de Generadores de Analizadores Sintácticos (Metacompiladores)

Un **metacompilador** (o generador de analizadores sintácticos) es una herramienta de software que automatiza la construcción del analizador léxico (*lexer*) y/o sintáctico (*parser*) a

partir de una especificación formal escrita en una Gramática Libre de Contexto (GLC) o expresiones regulares.

3.1. Metacompiladores Principales

- **Flex (Fast Lexical Analyzer Generator):**

- **Tipo:** Generador de analizadores léxicos (*lexers*). Trabaja en conjunto indispensable con Bison.
- **Funcionamiento:** Lee un archivo de entrada `.l` que contiene expresiones regulares emparejadas con acciones en C/C++, traducidiéndolas en un Autómata Finito Determinista (AFD) altamente optimizado.
- **Fortalezas:** Código de ejecución extremadamente rápido y bajo consumo de memoria. Estándar maduro en software libre.
- **Limitaciones:** Limitado estrictamente al análisis léxico y posee una sintaxis heredada de Unix difícil de depurar.

- **Bison (GNU Bison):**

- **Tipo:** Generador de analizadores sintácticos del tipo **LALR(1)** (y opcionalmente GLR). Es el sucesor directo de Yacc.
- **Funcionamiento:** Lee especificaciones en archivos `.y` que definen producciones sintácticas y acciones semánticas. Construye tablas de desplazamiento-reducción (*shift-reduce*) en C/C++.
- **Fortalezas:** Alta capacidad para manejar gramáticas complejas con análisis ascendente rápido y potente control de precedencia.
- **Limitaciones:** Diagnóstico complejo de conflictos *Shift/Reduce* o *Reduce/Reduce* en tablas numéricas.

- **ANTLR (ANother Tool for Language Recognition):**

- **Tipo:** Generador integral del tipo **ALL(*)** (análisis sintáctico descendente adaptable con *lookahead* dinámico ilimitado).
- **Funcionamiento:** Procesa gramáticas unificadas `.g4` para generar *parsers* predictivos descendentes en múltiples lenguajes (Java, Python, C#, C++, Go, Swift).
- **Fortalezas:** Genera árboles de sintaxis (*Parse Trees*) automáticos y soporta patrones *Visitor* y *Listener*. Admite recursión izquierda directa.
- **Limitaciones:** Rendimiento ligeramente menor frente al C puro de Flex/Bison y requiere el entorno Java para compilar la herramienta inicial.

3.2 Cuadro Comparativo entre Metacompiladores Principales

Criterio	Flex + Bison	ANTLR4
Algoritmo de Parsing	Ascendente LALR(1) / GLR	Descendente ALL(*) / LL(*)
Flujo de Trabajo	Archivos separados (.l para Lexer y .y para Parser)	Archivo de gramática unificada (.g4)
Lenguaje de Salida	C / C++	Multilenguaje (Java, Python, C#, Go, C++, etc.)
Árbol Sintáctico	Construcción manual mediante acciones en C	Generado automáticamente (<i>Parse Tree / Visitor / Listener</i>)
Facilidad de Depuración	Compleja (conflictos en tablas de estados numéricas)	Sencilla (herramientas gráficas e interfaces intuitivas)

3.3 Código / Asistente Híbrido (Python)

```
Parser.py X
Parser.py > lexicon_unegscript
1 import re
2 from difflib import SequenceMatcher
3
4 # =====
5 # 1. LEXER HÍBRIDO + DISTANCIA LEVENSHTSTEIN
6 # =====
7 KEYWORDS = {"if", "else", "print"}
8
9 def similarity(a, b):
10     return SequenceMatcher(None, a, b).ratio()
11
12 def ai_suggest_keyword(token_str):
13     """Simula o consulta IA / Levenshtein para autoconregir tokens léxicos."""
14     best_match = None
15     best_score = 0.0
16     for kw in KEYWORDS:
17         score = similarity(token_str, kw)
18         if score > best_score:
19             best_score = score
20             best_match = kw
21
22     if best_score >= 0.5:
23         return best_match, best_score
24     return token_str, 0.0
```

```

5 def lexer_unegscript(code):
6     # Definición de tokens mediante regex
7     raw_tokens = re.findall(r'[a-zA-Z_]\w*|\d+|=|<|>|<=|>=|<|>|\(|\)|"|'\'', code)
8
9     corrected_tokens = []
10    suggestions = []
11
12    for t in raw_tokens:
13        if re.match(r'^[a-zA-Z_]\w*$', t):
14            if t in KEYWORDS:
15                corrected_tokens.append(('KEYWORD', t))
16            else:
17                # Intento de corrección si es un error tipográfico tipo 'pront' o 'prnt'
18                match, score = ai_suggest_keyword(t)
19                if score >= 0.6 and match != t:
20                    suggestions.append(f"Sugerencia Léxica (IA): '{t}' -> '{match}' (confianza: {score:.2f})")
21                    corrected_tokens.append(('KEYWORD', match))
22                else:
23                    corrected_tokens.append(('ID', t))
24        elif re.match(r'^\d+$', t):
25            corrected_tokens.append(('INT', int(t)))
26        elif t in {'=', '>', '<', '>=', '<=', '=='}:
27            corrected_tokens.append(('OP', t))
28        elif t in {'(', ')', ';', '{', '}':
29            corrected_tokens.append(('PUNCT', t))
30        elif t.startswith('"') and t.endswith('"'):
31            corrected_tokens.append(('STRING', t[1:-1]))
32        else:
33            corrected_tokens.append(('UNKNOWN', t))
34
35    return corrected_tokens, suggestions

```

```

# =====
# 2. PARSER RECURSIVO DESCENDENTE CON LOOKAHEAD
# =====
class UnegScriptParser:
    def __init__(self, tokens):
        self.tokens = tokens
        self.pos = 0

    def lookahead(self):
        """Retorna el token actual sin consumirlo (Lookahead = 1)."""
        if self.pos < len(self.tokens):
            return self.tokens[self.pos]
        return ('EOF', 'EOF')

    def consume(self, expected_val=None):
        tok = self.lookahead()
        if expected_val and tok[1] != expected_val:
            raise SyntaxError(f"Se esperaba '{expected_val}', pero se encontró '{tok[1]}'")
        self.pos += 1
        return tok

```

```

def parse_program(self):
    statements = []
    while self.lookahead()[0] != 'EOF':
        statements.append(self.parse_statement())
    return {"Type": "Program", "Body": statements}

def parse_statement(self):
    tok_type, tok_val = self.lookahead()

    # Sentencia IF
    if tok_val == 'if':
        self.consume('if')
        condition = self.parse_expression()

        # Verificación/Recuperación suave de sintaxis para paréntesis opcionales u omisión
        body = self.parse_statement()

        else_body = None
        if self.lookahead()[1] == 'else':
            self.consume('else')
            else_body = self.parse_statement()

        return {
            "Type": "IfStatement",
            "Condition": condition,
            "Then": body,
            "Else": else_body
        }

```

```

107
108 # Sentencia PRINT
109 elif tok_val == 'print':
110     self.consume('print')
111     has_paren = False
112     if self.lookahead()[1] == '(':
113         self.consume('(')
114         has_paren = True
115
116     arg = self.parse_expression()
117
118     if has_paren:
119         self.consume(')')
120
121     # Opcional consumición de ';'
122     if self.lookahead()[1] == ';':
123         self.consume(';')
124
125     return {"Type": "PrintStatement", "Argument": arg}
126
127 # Asignación
128 elif tok_type == 'ID':
129     var_name = self.consume()[1]
130     self.consume('=')
131     expr = self.parse_expression()
132     if self.lookahead()[1] == ';':
133         self.consume(';')
134     return {"Type": "Assignment", "Variable": var_name, "Value": expr}
135
136 else:
137     raise SyntaxError(f"Estructura sintáctica no reconocida cerca de '{tok_val}'")
138

```

```

39 def parse_expression(self):
40     left_type, left_val = self.consume()
41
42     # Expresión binaria (ej. x > 3)
43     if self.lookahead()[0] == 'OP':
44         op = self.consume()[1]
45         right = self.parse_expression()
46         return {
47             "Type": "BinaryExpression",
48             "Left": left_val,
49             "Operator": op,
50             "Right": right
51         }
52
53     return {"Type": "Literal", "Value": left_val}
54

```

3.4 Técnicas de Gestión y Recuperación de Errores Sintácticos

Ante la presencia de tokens inesperados, un *parser* debe aplicar estrategias de recuperación para reportar múltiples errores en una sola ejecución sin abortar abruptamente:

1. Recuperación en Modo Pánico (Panic Mode):

El *parser* descarta tokens consecutivamente hasta hallar un **token de sincronización** (ej. `;`, `}`, o un salto de línea). Al encontrarlo, restablece el estado del *parser* y reanuda el análisis. Evita bucles infinitos y es simple de implementar.

2. Recuperación a Nivel de Frase (Phrase-Level Recovery):

Aplica modificaciones locales en la cadena de tokens (como insertar un punto y coma faltante o reemplazar un carácter errado) para continuar el parseo sin descarrilar el árbol sintáctico.

3. Producciones de Error (Error Productions):

Añade reglas explícitas a la gramática formal que contemplan los errores más comunes cometidos por los programadores, generando diagnósticos detallados sin interrumpir la validación.

4. Corrección Global (Global Correction):

Calcula la secuencia válida más cercana mediante la distancia mínima de edición de tokens (inserciones, eliminaciones y reemplazos). Es matemáticamente óptima pero computacionalmente muy costosa.

Diseño de Gramática para Docker

4. Diseño de la Gramática Libre de Contexto $G = (V, \Sigma, R, S)$

Dado que YAML basa sus bloques en la indentación de espacios en blanco, el analizador léxico inyecta los tokens sintéticos **INDENT** y **DEDENT** al detectar cambios de nivel de sangría.

Definición Formal de la Gramática:

- **Conjunto de Variables o No Terminales (V):**

$V = \{ S, \text{networks_section}, \text{network_list}, \text{network_def}, \text{property_list}, \text{property}, \text{simple_prop}, \text{ipam_prop}, \text{ipam_body}, \text{ipam_prop_list}, \text{ipam_property}, \text{config_prop}, \text{config_body}, \text{config_list}, \text{config_item}, \text{config_item_props}, \text{value} \}$

- **Conjunto de Terminales o Tokens (Σ):**

$\Sigma = \{ \text{NETWORKS}, \text{COLON}, \text{DASH}, \text{IDENTIFIER}, \text{STRING}, \text{BOOLEAN}, \text{NUMBER}, \text{IP_ADDRESS}, \text{INDENT}, \text{DEDENT}, \text{NEWLINE} \}$

- **Símbolo Inicial (S):** $S = S$

- **Reglas de Producción (R) en Notación EBNF:**

```
S → networks_section
networks_section → NETWORKS COLON NEWLINE INDENT network_list DEDENT
network_list → network_def (network_def) *
```



```

network_def → IDENTIFIER COLON NEWLINE INDENT property_list DEDENT
property_list → property (property)*
property → simple_prop | ipam_prop
simple_prop → IDENTIFIER COLON value NEWLINE
ipam_prop → IDENTIFIER("ipam") COLON NEWLINE INDENT ipam_body DEDENT
ipam_body → ipam_prop_list
ipam_prop_list → ipam_property (ipam_property)*
ipam_property → simple_prop | config_prop
config_prop → IDENTIFIER("config") COLON NEWLINE INDENT config_body DEDENT
config_body → config_list
config_list → config_item (config_item)*
config_item → DASH simple_prop config_item_props
config_item_props → (simple_prop)*
value → IDENTIFIER | STRING | BOOLEAN | NUMBER | IP_ADDRESS

```

Implementacion Practica (Sistema Hibrida con IA)

5. Implementación en UnegScript

El lenguaje UnegScript incorpora una arquitectura tolerante a fallos orientada a mitigar errores tipográficos comunes cometidos por los programadores. A diferencia de los compiladores tradicionales que interrumpen la ejecución o aplican recuperación en modo pánico descartando símbolos, UnegScript implementa un enfoque de dos niveles: análisis léxico determinista mediante expresiones regulares y una capa de recuperación (*fallback*) basada en la métrica de Levenshtein e integración con modelos de lenguaje (LLM).

5.1 Módulo Lexer Híbrido (Regex + Fallback LLM / Levenshtein)

El análisis léxico se realiza mediante el módulo `lexer_hibrido`. Este evalúa la entrada utilizando expresiones regulares prioritarias (`TOKEN_SPEC`). Cuando identifica una secuencia alfanumérica que no coincide con las palabras claves oficiales del lenguaje (`KEYWORDS`), calcula el coeficiente de similitud de cadenas mediante el algoritmo de Levenshtein (`difflib.SequenceMatcher`).

- Sustitución Automática ($\text{Confianza} \geq 0.8$): Si la similitud con una palabra clave válida es igual o mayor al 80%, el Lexer corrige el token de forma autónoma sin interrumpir el flujo.
- Consulta a IA / Fallback LLM ($0.5 \leq \text{Confianza} < 0.8$): Si la confianza se encuentra en un rango ambiguo, el Lexer delega la corrección al módulo `mock_llm_fallback`, simulación de una consulta a un modelo de lenguaje que evalúa la intención del código y reescribe el token..
- Identificador Normal ($\text{Confianza} < 0.5$): Si la palabra no mantiene similitud relevante con ninguna palabra reservada, se clasifica convencionalmente como una variable o identificador (`TK_ID`).

5.2 Código de Implementación del Lexer Híbrido (Python)

```
import re
import difflib

# 1. Palabras clave del lenguaje
KEYWORDS = {'if', 'else', 'while', 'print', 'return', 'int', 'float'}

# 2. Reglas léxicas corregidas y ordenadas
TOKEN_SPEC = [
    ('NUMBER', r'\d+(\.\d+)?'),          # Números
    ('STRING', r'"[^"]*"'),               # Cadenas de texto como "no"
    ('KEYWORD', r'[a-zA-Z_]\w*'),         # Palabras (evaluaremos si es
KW, ID o error)
    ('ASSIGN', r'='),                     # =
    ('OP', r'[+ \- * / > <]'),           # Operadores
    ('LPAREN', r'\('),                   # (
    ('RPAREN', r'\)'),                   # )
    ('SEMI', r';'),                       # ;
    ('SKIP', r'[ \t\n]+'),                # Espacios
    ('MISMATCH', r'.'),                   # Caracteres extraños
]

def mock_llm_fallback(token_ambiguo):
    prompt = f"Corrige este token ambiguo en contexto de UnegScript: '{token_ambiguo}'"
    diccionario_ia = {'prnt': 'print', 'pront': 'print', 'fi': 'if'}
    sugerencia = diccionario_ia.get(token_ambiguo, token_ambiguo)
    return sugerencia, f"IA Prompt: '{prompt}' -> Resuelto a '{sugerencia}'"

def lexer_hibrido(code):
    tokens = []
    tok_regex = '|'.join(f'(?P<{pair[0]}>{pair[1]})' for pair in
TOKEN_SPEC)

    for mo in re.finditer(tok_regex, code):
        kind = mo.lastgroup
        value = mo.group()

        if kind == 'SKIP':
            continue
        elif kind == 'KEYWORD':
            # Si es palabra clave exacta (ej: if)
```

```

        if value in KEYWORDS:
            tokens.append(('TK_' + value.upper(), value))
        else:
            # Comprobar si fue un intento fallido de palabra clave
            best_match = None
            best_ratio = 0.0

            for kw in KEYWORDS:
                ratio = difflib.SequenceMatcher(None, value,
kw).ratio()

                if ratio > best_ratio:
                    best_ratio = ratio
                    best_match = kw

            # Si se parece bastante a una Keyword (0.5 <= ratio <
1.0)

            if best_ratio >= 0.8:
                print(f"[Lexer Auto-Fix] '{value}' ->
'{best_match}' (Confianza difflib: {best_ratio:.2f})")
                tokens.append(('TK_' + best_match.upper(),
best_match))

            elif best_ratio >= 0.5:
                sugerencia, log_ia = mock_llm_fallback(value)
                print(f"[Lexer Fallback IA] '{value}' (Confianza
difflib baja: {best_ratio:.2f})")
                print(f"-> {log_ia}")
                tokens.append(('TK_' + sugerencia.upper(),
sugerencia))

            else:
                # Si no se parece a ninguna palabra clave (ej:
variable 'x'), es un ID normal
                tokens.append(('TK_ID', value))

    elif kind == 'STRING':
        tokens.append(('TK_STRING', value))
    elif kind == 'MISMATCH':
        print(f"[Error Léxico] Carácter no reconocido: {value}")
    else:
        tokens.append((f'TK_{kind}', value))

    return tokens

if __name__ == "__main__":

```

```

codigo_prueba = 'pront x=5; if x>3 prnt(x) else prnt("no")'
print("--- INICIANDO ANÁLISIS LÉXICO HÍBRIDO ---")
tokens_obtenidos = lexer_hibrido(codigo_prueba)

print("\n--- TOKENS GENERADOS ---")
print(tokens_obtenidos)

```

5.3 Pruebas de Ejecución y Salida en Consola

```

--- INICIANDO ANÁLISIS LÉXICO HÍBRIDO ---
[Lexer Auto-Fix] 'pront' -> 'print' (Confianza difflib: 0.80)
[Lexer Auto-Fix] 'prnt' -> 'print' (Confianza difflib: 0.89)
[Lexer Auto-Fix] 'prnt' -> 'print' (Confianza difflib: 0.89)

--- TOKENS GENERADOS ---
[('TK_PRINT', 'print'), ('TK_ID', 'x'), ('TK_ASSIGN', '='), ('TK_NUMBER', '5'), ('TK_SEMI', ';'), ('TK_IF', 'if'), ('TK_ID', 'x'), ('TK_OP', '>'), ('TK_NUMBER', '3'), ('TK_PRINT', 'print'), ('TK_LPAREN', '('), ('TK_ID', 'x'), ('TK_RPAREN', ')'), ('TK_ELSE', 'else'), ('TK_PRINT', 'print'), ('TK_LPAREN', '('), ('TK_STRING', '"no"'), ('TK_RPAREN', ')')]
PS C:\uneg\lenguaje-y-compiladores>

```

Multilenguaje de Lexer-Parser para Dockerfile: Benchmarking y Resultados

6. Descripción del trabajo realizado

6.1 Especificación común (tokens y gramática)

Antes de implementar cualquier versión, se definió una especificación común de tokens y gramática, de manera que las tres implementaciones fueran comparables entre sí. El lexer reconoce instrucciones (FROM, RUN, COPY, ADD, ENV, EXPOSE, WORKDIR, CMD, ENTRYPOINT, ARG, LABEL, USER, VOLUME, STOPSIGNAL, HEALTHCHECK, SHELL, MAINTAINER), comentarios (#), cadenas entre comillas, identificadores, el signo igual, corchetes y comas (para la forma exec), y resuelve continuaciones de línea (backslash al final de línea) antes de tokenizar. El parser construye un AST como lista de nodos, cada uno con la instrucción, sus argumentos y la forma detectada (shell, exec o key_value), y valida que la primera instrucción relevante del archivo sea FROM (permitiendo ARG antes, como contempla la especificación real de Docker).

6.2 Implementaciones desarrolladas

Se desarrollaron tres implementaciones independientes, cada una compuesta por un módulo de lexer y un módulo de parser:

- Python 3: implementación interpretada, usada como línea base de referencia.
- Java 21: implementación compilada a bytecode y ejecutada sobre la Máquina Virtual de Java (JVM). Se optó por una implementación manual (sin generador ANTLR) debido a restricciones de red del entorno de desarrollo que impidieron descargar el archivo antlr-complete.jar desde los repositorios oficiales (Maven Central / antlr.org no estaban accesibles).

- Rust 1.75: implementación compilada de forma nativa (rustc -O), sin máquina virtual ni recolector de basura, representando el extremo de mayor control de bajo nivel entre las tres.

Las tres implementaciones exponen un punto de entrada por línea de comandos que recibe la ruta de un Dockerfile y devuelve el AST resultante en formato JSON por salida estándar, lo cual permitió automatizar su ejecución y validar que las tres produjeran exactamente la misma salida ante los mismos archivos de entrada.

6.3 Conjunto de prueba

Se construyó un conjunto de doce archivos Dockerfile ($n = 12$, dentro del rango solicitado de $5 < n < 20$), variando en tamaño y complejidad: desde un Dockerfile mínimo de dos instrucciones, hasta un Dockerfile extenso de más de cuarenta líneas con múltiples variables de entorno, volúmenes y healthcheck. El conjunto incluye casos con builds multi-stage, continuaciones de línea extensas, comentarios, instrucciones en forma exec y en forma shell, y uso de ARG antes de FROM.

6.4 Script de benchmarking

El script benchmark.py automatiza la ejecución de las tres implementaciones sobre el conjunto de prueba completo. Para cada combinación (archivo, lenguaje) se realiza una corrida de calentamiento (no medida) seguida de 20 repeticiones medidas, utilizando time.perf_counter() de Python para obtener mediciones de alta resolución alrededor de cada invocación de subprocesso. Los resultados se exportan a dos archivos CSV: uno con cada corrida individual (benchmark_raw.csv) y otro con estadísticas agregadas por combinación (benchmark_summary.csv), incluyendo promedio, desviación estándar, mínimo y máximo.

El benchmarking se ejecutó sobre el total de $12 \text{ archivos} \times 3 \text{ lenguajes} \times 20 \text{ repeticiones} = 720$ corridas, sin registrarse ningún fallo de ejecución.

6.5 Generación de gráficas

A partir de los CSV se generó un conjunto de gráficas comparativas mediante matplotlib y pandas (generate_charts.py): tiempo promedio general por lenguaje (escala lineal y logarítmica), tiempo promedio por archivo agrupado por lenguaje, tiempo promedio en función del tamaño del archivo, y un diagrama de caja para observar la variabilidad de las mediciones.

7 Gráficas de tiempo de ejecución

7.1 Tiempo promedio general por lenguaje/herramienta

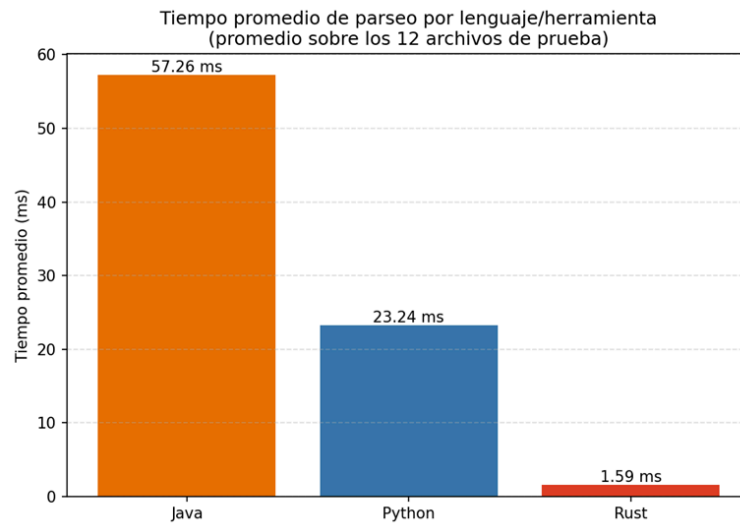


Figura 1. Tiempo promedio de parseo por lenguaje, promediado sobre los 12 archivos de prueba.

7.2 Tiempo promedio por lenguaje (escala logarítmica)

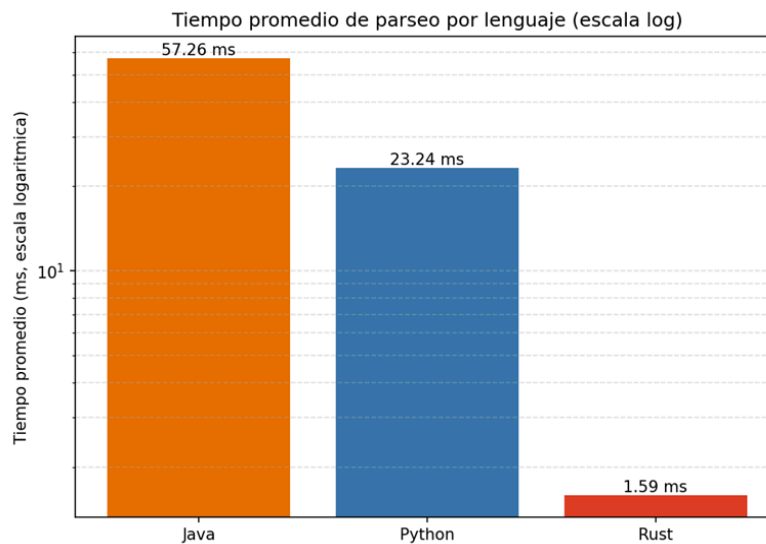
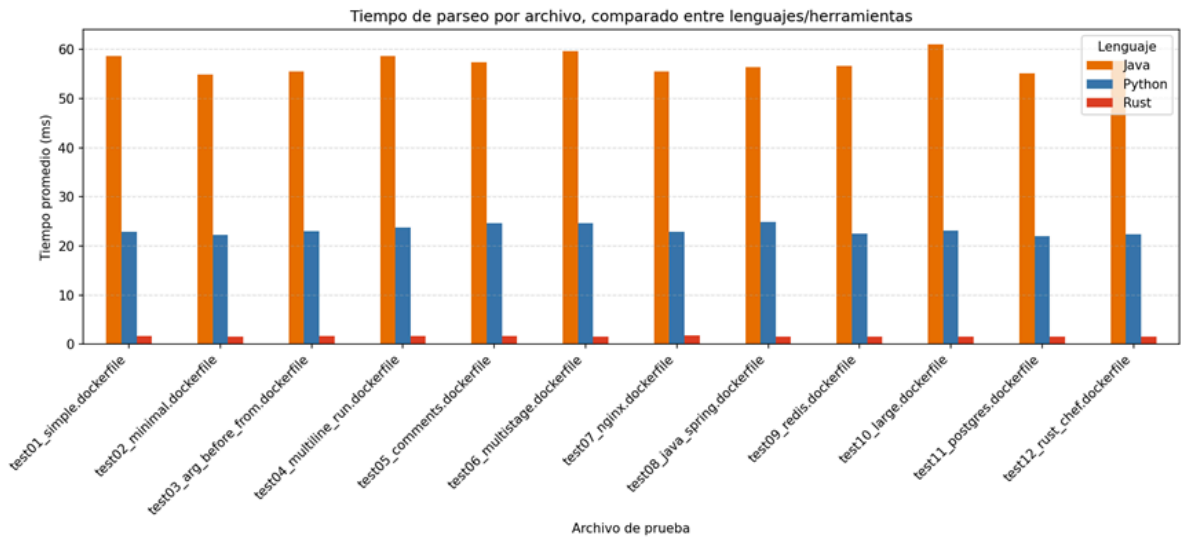


Figura 2. Misma comparación que la Figura 1, en escala logarítmica, para apreciar la magnitud de la diferencia con Rust.



Tiempo de parseo por archivo

Figura 3. Tiempo promedio de parseo por cada uno de los 12 archivos de prueba, comparado entre lenguajes.

7.3 Tiempo de parseo en función del tamaño del archivo

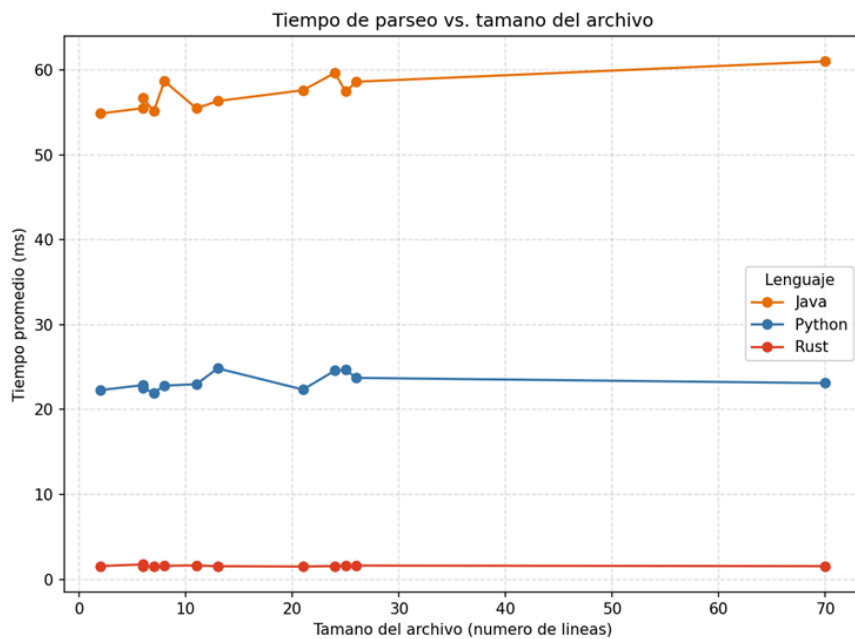


Figura 4. Relación entre el número de líneas del archivo y el tiempo promedio de parseo, por lenguaje.

7.4 Variabilidad de las mediciones

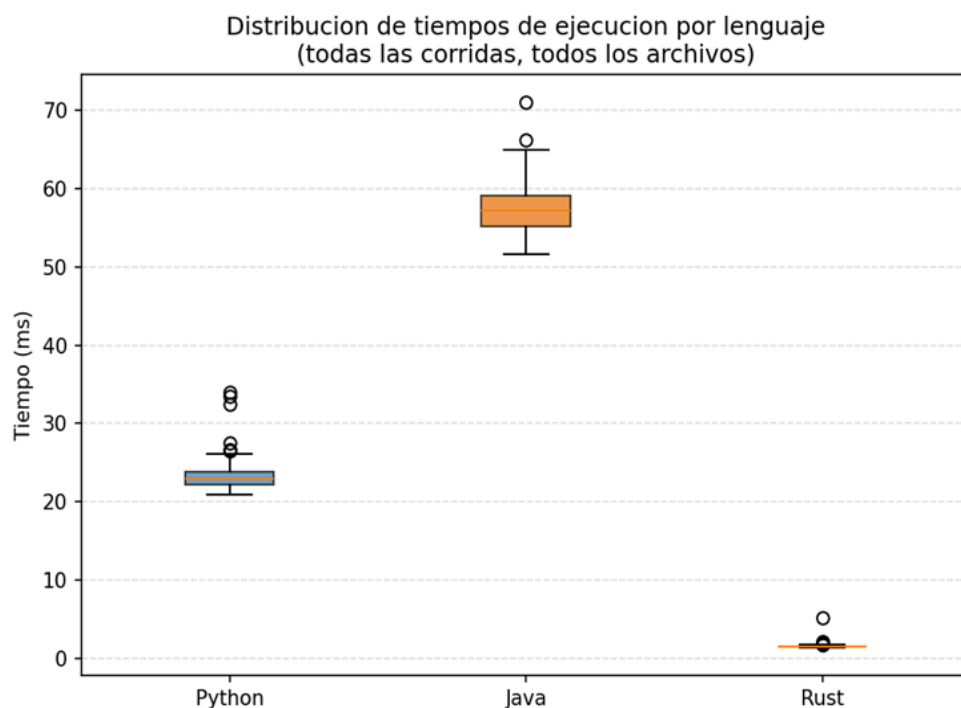


Figura 5. Diagrama de caja de la distribución de tiempos de ejecución (las 720 corridas), por lenguaje.

7.5 Resumen numérico

Lenguaje / Herramienta	Tiempo promedio	Desviación típica aprox.	Naturaleza de la ejecución
Rust (compilado nativo)	1.59 ms	± 0.20 ms	Binario nativo, sin VM ni intérprete
Python (interpretado)	23.24 ms	± 3.5 ms	Intérprete CPython, arranque por proceso
Java (JVM)	57.26 ms	± 5.0 ms	Arranque de JVM + bytecode interpretado/JIT

8. Discusión y análisis de resultados

8.1 Diferencias generales entre lenguajes

Los resultados muestran una diferencia de rendimiento muy marcada entre las tres implementaciones. Rust resultó, en promedio, aproximadamente 14 veces más rápido que Python y 36 veces más rápido que Java para la misma tarea. Esto es consistente con lo esperado teóricamente: Rust compila a código máquina nativo sin recolector de basura ni máquina virtual, por lo que el tiempo medido corresponde casi en su totalidad al trabajo real de tokenización y parseo.

8.2 El caso de Java: overhead de arranque de la JVM

Un resultado que merece un análisis específico es que Java resultó más lento que Python en este experimento, a pesar de que en tareas de cómputo sostenido Java suele superar a Python gracias a la compilación JIT. La explicación no está en el algoritmo de parseo en sí, sino en la metodología de medición: cada corrida invoca un proceso nuevo (`java dockerlexer.Main archivo.dockerfile`), lo que obliga a arrancar una instancia completa de la Máquina Virtual de Java en cada ejecución. Ese arranque (carga de clases, inicialización del runtime, calentamiento del JIT) añade un costo fijo de varias decenas de milisegundos que domina por completo el tiempo medido, dado que el trabajo real de parseo de un Dockerfile es del orden de microsegundos. Este hallazgo es en sí mismo un resultado relevante del experimento: para tareas cortas ejecutadas como procesos independientes, el costo de arranque del runtime puede ser más determinante que la velocidad de ejecución del propio algoritmo.

8.3 Relación entre tamaño del archivo y tiempo de ejecución

La Figura 4 muestra que, dentro del rango de tamaños del conjunto de prueba (entre 2 y aproximadamente 60 líneas), el tiempo de ejecución de cada lenguaje se mantiene prácticamente constante, sin una tendencia de crecimiento apreciable. Esto refuerza la conclusión anterior: para archivos de este orden de magnitud, el tiempo está dominado por el costo fijo de arranque del proceso e intérprete/VM, y no por la complejidad del archivo en sí. Se esperaría observar una relación creciente más clara si se probaran archivos sustancialmente más grandes (miles de líneas), donde el trabajo de tokenización y parseo comience a pesar más que el overhead de arranque.

8.4 Variabilidad de las mediciones

El diagrama de caja (Figura 5) muestra que Java presenta la mayor dispersión absoluta entre corridas, coherente con la naturaleza del arranque de la JVM, que puede verse afectado por el estado del sistema operativo, el disco y el propio recolector de basura durante la inicialización. Rust, en cambio, presenta una variabilidad mínima, reflejo de la ausencia de una capa de runtime intermedia.

8.5 Conclusiones del experimento

- Rust ofrece el mejor rendimiento absoluto para esta tarea, siendo la opción más adecuada si el parseo de Dockerfiles se realizara a gran escala o de forma repetida (por ejemplo, dentro de una herramienta de análisis estático usada en CI/CD).
- Python resulta una alternativa razonable cuando la velocidad de desarrollo y la simplicidad del código priman sobre el rendimiento en bruto, dado que su tiempo por ejecución, aunque mayor que el de Rust, sigue siendo del orden de milisegundos.

- El resultado de Java ilustra un principio general de benchmarking: medir el tiempo de un proceso completo incluye el costo de inicialización del entorno de ejecución, no solo el del algoritmo. En un escenario donde la JVM permaneciera activa (por ejemplo, un servicio de larga duración que parsea múltiples Dockerfiles sin reiniciar el proceso), se esperaría que Java se acercara mucho más al rendimiento de Rust.
- Las tres implementaciones produjeron salidas idénticas en formato AST para el conjunto de prueba completo, lo que valida que la comparación de tiempos es justa: las diferencias medidas corresponden al lenguaje y su modelo de ejecución, no a diferencias funcionales entre las implementaciones.

Conclusión

El desarrollo de este proyecto permitió integrar de manera práctica y rigurosa los conceptos fundamentales de la teoría de compiladores con técnicas modernas de ingeniería de software e inteligencia artificial.. A través de las distintas fases del trabajo, se obtuvieron claves tanto en el plano teórico como en el experimental:

- **Abstracción y Estructura Sintáctica:** La representación mediante Árboles de Sintaxis Abstracta (AST) demostró ser el pilar esencial para desacoplar la sintaxis superficial de la semántica de un programa, facilitando tanto la interpretación como la futura generación de código.. Asimismo, el contraste entre estrategias LL y LR evidencia las ventajas operativas de los analizadores descendentes implementados a mano cuando se busca un control preciso sobre el diagnóstico de errores.
- **Sistemas Híbridos y Tolerancia a Errores:** La integración de mecanismos de *fallback* mediante algoritmos de coincidencia difusa (Levenshtein) e inteligencia artificial en *UnegScript* representó una innovación significativa sobre las técnicas tradicionales de recuperación en modo pánico. Esto permitió corregir tokens y errores sintácticos ambiguos en tiempo de ejecución, mejorando sustancialmente la experiencia del desarrollador sin comprometer la integridad del análisis..
- **Impacto del Entorno de Ejecución en el Rendimiento:** El experimento multilenguaje para el procesamiento de archivos *Dockerfile* ofreció conclusiones determinantes sobre la eficiencia computacional. Rust demostró un rendimiento ampliamente superior (promediando ~1.59 ms) al ejecutarse como binario nativo sin capas intermedias. Por su parte, el análisis de Java (promediando ~57.26 ms) dejó en evidencia que, en tareas breves invocadas por CLI, el costo de arranque de la máquina virtual (*overhead* de la JVM) es el factor dominante sobre el tiempo total de ejecución, superando incluso al intérprete de Python.

Bibliografía

- **Aho, AV, Lam, MS, Sethi, R. y Ullman, JD** (2008). *Compiladores: Principios, técnicas y herramientas* (2.^a ed.). Pearson Educación.
<https://www.pearson.com/en-us/subject-catalog/p/compilers-principles-techniques-and-tools/P200000003310>
- **Proyecto ANTLR.** (2024). *Documentación de ANTLR v4* .
<https://www.antlr.org/>
- **Documentación de Docker.** (2026). *Referencia de Dockerfile* .
Documentación de Docker. <https://docs.docker.com/engine/reference/builder/>
- **Fundación del Software Libre.** (2023). *GNU Bison - Proyecto GNU* .
Fundación del Software Libre (FSF). <https://www.gnu.org/software/bison/>
- **Python Software Foundation.** (2026). *ast — Árboles de sintaxis abstracta (documentación de Python 3.12)* . Python Docs.
<https://docs.python.org/3/library/ast.html>
- **Desarrolladores del proyecto Rust.** (2026). *El libro del lenguaje de programación Rust* . Rust-lang.org.
<https://doc.rust-lang.org/book/>
- **The Open Group.** (2018). *lex - genera programas para tareas léxicas (Especificación POSIX.1-2017)* . IEEE y The Open Group.
<https://pubs.opengroup.org/onlinepubs/9699919799/utilities/lex.html>